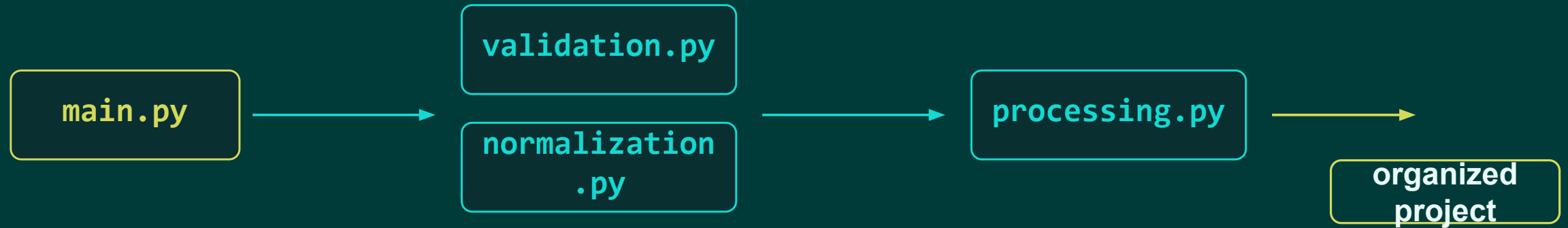


Creating Your Own Modules

Day 5 • Block 38 • 30 minutes teaching + 30 minutes exercises



Key shift

Yesterday we wrote reusable functions. Today we put them where future programs can reuse them.

What Students Should Be Able to Do

A module is just a Python file that other Python files can import.

- 1 Explain how Python files can become modules
- 2 Split a long program across focused files
- 3 Import functions from a custom module
- 4 Choose clear module names and responsibilities
- 5 Recognize circular-import warning signs
- 6 Use `if __name__ == "__main__":` at a basic level

Session shape

30 minutes: concepts and examples. 30 minutes: refactor one file into several modules.

From One Large File to Several Focused Files

A working script can become hard to navigate as it grows.

```
processor.py
├── read files
├── normalize values
├── validate records
├── classify records
├── create summary
└── write output
```

200+ lines



```
project/
main.py
validation.py
normalization.py
processing.py
reporting.py
```

Mental model

Split by responsibility, not by line count alone.

A Module Is a Python File With Reusable Code

The filename becomes the module name when another file imports it.

validation.py →

```
def is_valid_percentage(value: int) -> bool:  
    return 0 <= value <= 100  
  
def is_valid_record_count(value: int) -> bool:  
    return value > 0
```

Important

Nothing magical happened. We put related functions in a named Python file.

When `main.py` imports `validation.py`, those functions become available to `main.py`.

Import Your Own Functions

Custom modules use the same import idea as standard-library modules.

```
# main.py

from validation import (
    is_valid_percentage,
    is_valid_record_count,
)

completion = 95
record_count = 12000

if is_valid_percentage(completion):
    print("Completion is valid")
```

```
from validation import ...
```

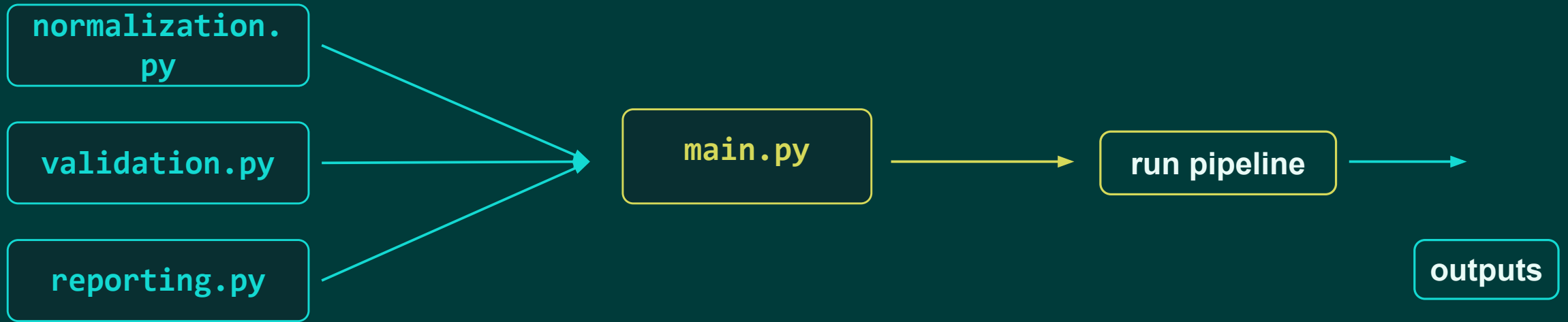
Read as: import these names from validation.py.

Beginner habit

Keep the import near the top of the file so readers know what this file depends on.

Module Mental Model

Each file owns a responsibility; main.py coordinates the workflow.



Rule of thumb

Support files provide capabilities. main.py describes the order of the work.

Why Split Code?

The goal is not more files. The goal is easier thinking.

Easier navigation

Find validation logic without scanning everything.

Reusable logic

Use the same functions in another project.

Easier testing

Test validation without running the whole pipeline.

Clear responsibilities

Each file answers one kind of question.

Split code when separation makes the program easier to understand, change, reuse, or test.

Do Not Put Every Function in a Separate File

Too many tiny files can be harder to understand than one organized file.

```
project/  
  normalize_record_id.py  
  normalize_status.py  
  validate_record.py  
  calculate_summary.py  
  print_summary.py
```

Too fragmented

```
project/  
  normalization.py  
  validation.py  
  reporting.py  
  main.py
```

Grouped by responsibility

Target

Group related functions together so the file name communicates the purpose.

Choose Module Names That Explain Responsibility

A good filename tells readers what kind of code belongs inside.

validation.py

Is this data acceptable?

normalization.py

How should raw values be cleaned?

processing.py

How should one record be transformed?

reporting.py

How should results be summarized?

```
# Less helpful names
stuff.py
helpers.py
misc.py
things.py
```

Generic names often hide mixed responsibilities.

Example Responsibility Map

Start with the questions each file should answer.

normalization.py

Clean raw text values

- Strip spaces
- Change case
- Standardize IDs

validation.py

Decide if data is acceptable

- Required fields
- Allowed values
- Ranges

processing.py

Transform records

- Parse numbers
- Classify status
- Build output

main.py

Coordinate workflow

- Load data
- Call stages
- Write output

A Small Multi-File Project

This is enough structure for today's exercise.

```
day5_project/  
  main.py  
  normalization.py  
  validation.py  
  processing.py  
  reporting.py
```

```
# main.py imports project functions  
from normalization import normalize_status  
from validation import validate_record  
from processing import process_record  
from reporting import create_summary
```

Key idea

The files sit side by side in the same project folder, so main.py can import them by module name.

Refactor by Moving Functions Intentionally

Do not move everything at once. Move one responsibility, then run the program.



```
# main.py
from normalization import normalize_status

status = normalize_status(" ACTIVE ")
```

Small moves make errors easier to isolate.

Demo Before: One File Does Everything

The code works, but all responsibilities are mixed together.

```
# processor.py

def normalize_status(status):
    return status.strip().lower()

def validate_record(record):
    if record["record_id"].strip() == "":
        return "rejected"
    return "ready"

records = [...]

for record in records:
    status =
    normalize_status(record["status"])
    result = validate_record(record)
    print(status, result)
```

Mixed concerns

Definitions, data, workflow, and output all live in the same file.

Refactor goal

Keep the main program readable while preserving behavior.

BLOCK 38

Demo After: Functions Move Into Modules

The workflow stays in main.py. The details move into focused files.

```
# normalization.py
```

```
def  
normalize_status(status):  
    return  
status.strip().lower()
```

```
# validation.py
```

```
def validate_record(record):  
    if  
record["record_id"].strip()  
== "":  
        return "rejected"  
    return "ready"
```

```
# main.py
```

```
from normalization import normalize_status  
from validation import validate_record  
  
for record in records:  
    status = normalize_status(record["status"])  
    result = validate_record(record)  
    print(status, result)
```

Importing a Module Executes the File Once

Python loads the module so its functions and names become available.



```
# validation.py

def is_valid_percentage(value):
    return 0 <= value <= 100

print("Loaded validation")
```

```
# main.py

import validation

# "Loaded validation" prints during
import
```

Caution

Avoid top-level demonstration code in reusable modules unless you intentionally protect it.

BLOCK 38

__name__ and __main__

Use this pattern to separate reusable code from program startup behavior.

```
def main():  
    print("Starting processing")  
  
if __name__ == "__main__":  
    main()
```

When run directly: `__name__ == __main__`

When imported: `__name__ == module name`

Beginner translation

Run main() only when this file is the program's starting point.

Why the `__main__` Guard Matters

Reusable modules should not unexpectedly run a whole program when imported.

```
# validation.py

def is_valid_record_count(value):
    return value > 0

# Demo code protected here
def main():
    print(is_valid_record_count(10))

if __name__ == "__main__":
    main()
```

```
# main.py

from validation import
is_valid_record_count

# Import gets the function.
# The validation demo does not run.
```

**Reusable code stays reusable.
Program startup stays obvious.**

Circular Imports Are a Warning Sign

A circular import usually means responsibilities are tangled.



```
# module_a.py
from module_b import clean_record

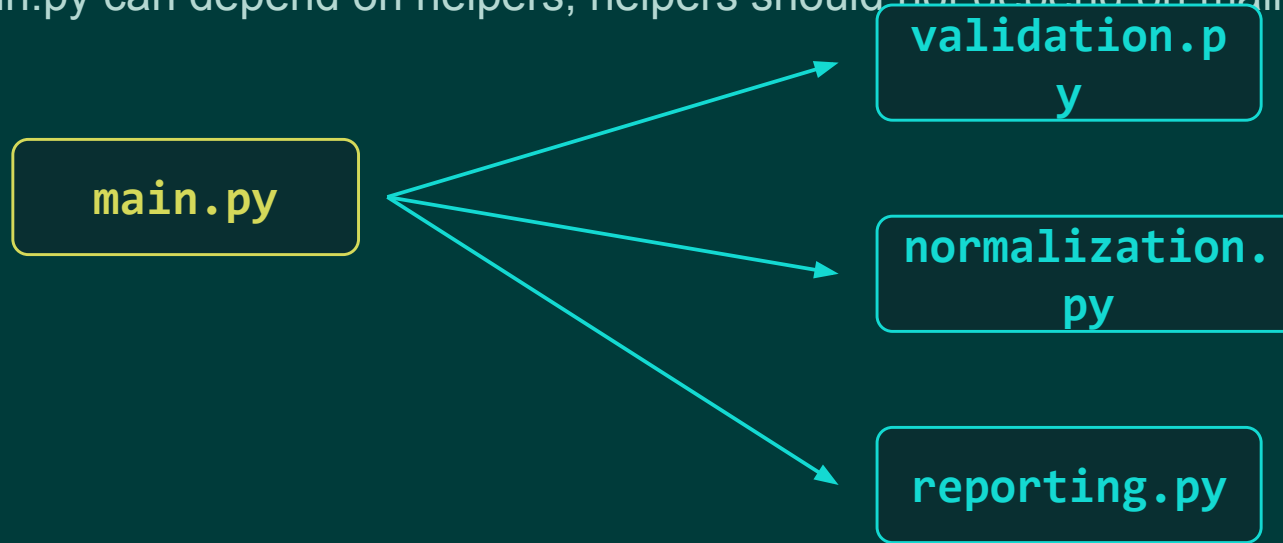
# module_b.py
from module_a import validate_record
```

Practical response

Move shared logic into a third module or rethink which file should own the responsibility.

Prefer One-Way Dependencies

main.py can depend on helpers; helpers should not depend on main.py.



Healthy direction

main.py knows about helper modules. Helper modules do not need to know how the whole program starts.

Common Import Mistakes

Most beginner import errors are naming, folder, or spelling problems.

Wrong filename

validation.py but
import validations

Wrong function name

validate_record vs
valid_record

File not beside main.py

module not found in
project folder

Circular dependency

two modules import
each other

Debugging habit

Read the error name first: ModuleNotFoundError usually means Python cannot find the module.

Guided Exercise: Refactor One File into Modules

30 minutes • turn a working script into a small project.

Starting point:

```
processor.py
  normalize_record_id()
  normalize_status()
  validate_record()
  process_record()
  create_summary()
  main loop
```



Target:

```
normalization.py
validation.py
processing.py
reporting.py
main.py
```

Exercise goal

Preserve the program's behavior while making responsibilities clearer.

Exercise 1: Create the Project Files

Timing: 5 minutes

5 minutes

```
day5_modules/  
  main.py  
  normalization.py  
  validation.py  
  processing.py  
  reporting.py
```

```
# main.py  
  
# Start with imports later.  
# For now, keep this file empty or add:  
  
print("Project files created")
```

Task: create the files side by side in the same folder.

Exercise 2: Move Normalization Functions

Timing: 7 minutes

7 minutes

```
# normalization.py

def normalize_record_id(record_id):
    return record_id.strip().upper()

def normalize_status(status):
    return status.strip().lower()
```

```
# main.py

from normalization import (
    normalize_record_id,
    normalize_status,
)

print(normalize_record_id(" eq-1001 "))
print(normalize_status(" ACTIVE "))
```

Run main.py after this step before moving more code.

Exercise 3: Move Validation and Processing

Timing: 10 minutes

10 minutes

```
# validation.py

def validate_record(record):
    if record["record_id"] == "":
        return "rejected", "Missing ID"

    return "ready", "Record passed"
```

```
# processing.py

from validation import validate_record

def process_record(record):
    status, message =
    validate_record(record)
    record["processing_status"] = status
    record["message"] = message
    return record
```

Checkpoint

After moving each file, import it from main.py and run a tiny test call.

Exercise 4: Build main.py as the Coordinator

Timing: 8 minutes

8 minutes

```
# main.py

from normalization import normalize_record_id
from processing import process_record
from reporting import create_summary

def main():
    records = [
        {"record_id": " eq-1001 ", "status": " ACTIVE "},
    ]

    processed = []

    for record in records:
        record["record_id"] = normalize_record_id(
            record["record_id"]
        )
        processed.append(process_record(record))
```

Applied Challenge: Convert the Morning Pipeline

Use modules to organize a file-based record-processing program.

Minimum structure:

```
day5_project/  
  main.py  
  file_io.py  
  validation.py  
  transformations.py  
  reporting.py
```

Required outcome

Move the morning CSV / JSON pipeline into focused modules. main.py should read like an outline of the workflow.

Success check

Another student should be able to open main.py and understand the program flow in under one minute.

Knowledge Check

Students should be able to answer these before moving on.

What makes a .py file usable as a module?

Why might validation.py be separate from main.py?

What does from validation import validate_record mean?

Why can circular imports be a design smell?

What does if __name__ == "__main__" protect?

What should main.py normally be responsible for?

Transition

Next, students will connect this organization to dependencies and project environments.